

Bachelor Thesis

Degree in Artificial Intelligence

Development of a Reinforcement Learning-Compatible Environment for Autonomous Drone Training in LiftOff Simulator

Juan Echevarria Echeverria

Advisors

Igor Rodriguez Rodriguez
Inigo Lopez Gazpio

June 18, 2025

Acknowledgements

Eskerrak eman nahi izanez gero, hemen idatzi testua. En caso de querer añadir agradecimientos, escribir aquí el texto.

Atal hau nahi ez baduzu, *main.tex* fitxategian komentatu lerro hori. En caso de no querer este apartado, comentalo en el fichero *main.tex*.

Abstract

This thesis presents the development of a reinforcement learning (RL)-compatible environment for training autonomous drone control agents within the high-fidelity drone simulator *Liftoff*. Unlike traditional research-oriented simulators, *Liftoff* is a commercial Steam-based game designed for human users, posing unique challenges for real-time programmatic control and state extraction.

To address this, a custom bridge was implemented that emulates gamepad input and captures visual feedback from the simulation, enabling seamless bidirectional communication between the agent and the environment. The resulting system is fully compatible with Gymnasium and supports training with standard RL libraries such as Stable Baselines3.

Baseline experiments were first conducted in simpler environments like Lunar Lander to evaluate different RL algorithms. These were followed by training and evaluation of agents (PPO, A2C, DDPG, TQC) in the integrated *Liftoff* environment, showing the feasibility of learning stable flight control policies. Additionally, a hybrid experience buffer was proposed to incorporate human flight data into training, improving early performance and convergence stability.

To the best of our knowledge, this is the first work to integrate a commercial Steam simulator with Gym-based RL agents for autonomous control. The project demonstrates that realistic, game-based simulations can be repurposed for AI research, offering a scalable and low-cost platform for developing and evaluating drone navigation strategies under near-realistic conditions.

Sustainable Development Goals

This project contributes to several of the Sustainable Development Goals (SDGs), particularly those related to innovation, sustainable infrastructure, and environmental protection. By developing a bridge between Reinforcement Learning agents and a high-fidelity drone simulator, the project enables scalable, low-risk training of autonomous drones. This directly supports **SDG 9: Industry, Innovation and Infrastructure**, by advancing research in robotics and artificial intelligence and facilitating the development of resilient, smart infrastructure systems. Furthermore, autonomous drones trained in realistic simulations can be deployed in applications such as infrastructure inspection, urban planning, and disaster response, aligning with **SDG 11: Sustainable Cities and Communities**. The simulation-based approach also promotes sustainability by reducing the need for physical drone trials, which consume energy, materials, and may pose safety risks—supporting **SDG 13: Climate Action** through resource-efficient innovation. Additionally, autonomous drones have proven value in environmental monitoring and agriculture, contributing to **SDG 15: Life on Land**. Finally, the tools developed in this project can be used for educational purposes, democratizing access to advanced robotics and AI research environments and supporting **SDG 4: Quality Education**. Overall, the project exemplifies how technological advancement can support both human development and environmental sustainability.

Contents

Contents	vii
List of Figures	ix
List of Tables	x
1 Introduction	1
1.1 Motivation	2
1.2 Objectives	2
2 Project Planning	5
2.1 Project Scope	5
2.2 Project Timeline: Gantt Chart	7
2.3 Risk Management	7
2.4 Monitoring and Control	9
3 Background and Tools	11
3.1 Reinforcement Learning	11
3.1.1 Introduction	11
3.1.2 Curriculum Learning	14
3.1.3 Imitation Learning	15
3.2 Python libraries for Reinforcement Learning	15
3.2.1 Gymnasium	15
3.2.2 UBC_GameIntelligence	17
3.2.3 Stable Baselines3	17
4 Related Work	21
5 Preliminary Experiments in Basic Environments	25
5.1 The Lunar Lander environment	25
5.2 Setup and training of agents	28
5.3 Metrics and baseline results	29
5.4 Lessons learned	29

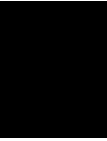
6	Integration of LiftOff with Gymnasium	33
6.1	Integration requirements for Liftoff	33
6.2	Overview of the integration	34
6.3	Communication System	35
6.3.1	Gamepad Emulation Module	36
6.3.2	Video Capturing Module	37
6.4	Environment Design	39
6.4.1	Reward wrappers	40
6.4.2	Action wrappers	40
6.4.3	Observation wrappers	41
6.4.4	Other wrappers	41
7	Experiments in the Custom Environment	43
7.1	Train setup and first try with PPO	43
7.1.1	Setup and training of the agent	43
7.1.2	Results and lessons learned	43
7.2	Training an A2C agent	44
7.2.1	Setup and training of the agent	44
7.2.2	Results and lessons learned	44
7.3	Using an off-policy agent (DDPG)	44
7.3.1	Setup and training of the agent	44
7.3.2	Results and lessons learned	45
7.4	Adding recorded data to the training	45
7.4.1	Setup and training of the agent	45
7.4.2	Results and lessons learned	46
7.5	Adding road and speed rewards	46
7.5.1	Setup and training of the agent	46
7.5.2	Results and lessons learned	47
7.6	Overall conclusions of the experiments	47
8	Conclusions and Future Work	49
	Appendix	53
	Bibliography	55

List of Figures

2.1	Gantt chart showing the timeline of the project tasks	7
3.1	Communication between an RL agent and the environment	12
3.2	MultiDiscreteDQN schema M: Number of discrete values for each actions N: Possible options for each discrete values	18
4.1	Images of the 3 considered simulators	23
5.1	Lunar lander environment image	26
6.1	Schema of the bridge and communication between the agent and Liftoff.	35
6.2	Drone movement and corresponding radio controller inputs.	36
6.3	Image of Liftoff simulation	38
6.4	Image obtained from Canny edge detection	38
7.1	Reward and episode reward evolutions on TQC agent training	46

List of Tables

2.1	Estimated effort by task group (Total: 300 hours)	6
2.2	Estimated effort by task group vs actual effort taken	10
4.1	Summary comparison of Velocidrone, Uncrashed, and Liftoff simulators.	23
5.1	Performance of UBC RL agents on Lunar Lander across different reward configurations and training durations (mean $\pm$ std reward).	30
5.2	Performance of SB3 RL agents on Lunar Lander with a discrete or continuous action spaces across different reward configurations and training durations (mean $\pm$ std reward).	31
6.1	Joystick Axis Mapping to Events and Liftoff Channels	37



Introduction

Autonomous navigation is one of the main challenges and research lines in the fields of artificial intelligence (AI) and modern robotics. Its use is highly relevant in different systems such as autonomous vehicles, mobile robots or drones, which all require advanced abilities to sense and understand their environment and update in real time to be able to traverse dynamic environments. In these tasks, it is a must for the agent to be able to adapt to how the environment evolves and the sudden changes it may present, asserting a safe and flexible behaviour.

In this context, Reinforcement Learning (RL) has been shown to be an extremely powerful tool. By interaction with the environment and using a reward-penalty system, the agent is able to learn in an autonomous way different control strategies that allow it to perform hard and complex tasks such as the take-off, navigation, or landing of a drone, as well as many other tasks autonomous systems may face, even under conditions and situations that have not been faced previously. This ability to learn without explicit supervision makes RL a truly appealing approach for autonomous systems, in general.

However, training these agents in physical environments presents concerns concerning limitations. Risks involving accidents, high operational and material costs and the impossibility to perform the millions of iterations necessary to achieve a successful learning make it impossible to use solely hardware in the initial steps of the training. As a solution to this, real simulators allow massmassive experiments to be performed in a secure and controlled way, reducing costs and needed time. Although this seems a great getaway to the problem, it carries a new obstacle to tackle, the gap that can be found in between the simulation and reality, the sim-to-real gap.

The sim-to-real gap refers to the discrepancy in between the behaviour of the agent in the simulated environment and a real one. This problem is mostly relevant in the case on drones, where many complex physical factors such as turbulences, sensor noise, the weight distribution or even the battery level of the drone, which are extremely hard to compute into

a simulation affect the responsiveness and ability to fly. If the simulator did not to implement all these phenomena reliably, the agents could learn strategies that are not suitable for the real world. That is why reducing this gap is a key need to develop consistent autonomous navigation systems which can be applied outside of the virtual environment.

1.1 Motivation

In the last few years, the interest to automatize drone flights has increased notoriously. This has happened in both, the industrial and academical worlds, pushing advancements in tasks such as agricultural monitoring [1], logistics [2] or even rescue tasks [3]. However, the complexity of this systems, added to the costs and risks involved when working with real hardware has limited the practical use of RL to simplified and control environments. On the other hand, alongside this an active community has arisen around drone simulators that were originally developed as video games, which have rapidly evolved up to offering a high fidelity physics system that highly resembles reality. The convergence in between the realistic simulation and the accessibility to new technology opens new opportunities for the investigation in autonomous control without needing for a high cost infrastructure.

This Final Degree Project aims to reduce this barrier, offering an accessible tool to train RL agents in a realistic simulated environment. To do so, the drone simulator Liftoff is used [4], widely used in the video game community and by (semi)professional pilots, specially in tasks like freestyle and drone racing, thanks to its physical fidelity and customizing possibilities. Inside this context, the pilot maneuvers the drone using a radio-control gamepad, which allows it to replicate the precision of real flights inside the simulation. This work aims to create a bridge in between the simulator and the standard environments in which RL is used, such as the ones defined by Gymnasium [5]. This connection would allow for experimentation with advanced learning algorithms without the need to utilize real hardware in the early stages of the development, easing the research and validation in conditions closer to reality.

1.2 Objectives

The main goal of this project is the development of a tool which allows training RL agents in the realistic simulator of drones Liftoff, using a custom made environment compatible with Gymnasium.

To achieve this general goal, the following specific objectives are defined:

1. Conducting a research of the state of the art in Reinforcement Learning and its application in autonomous drone flights, including the exploration of the different existing simulators, aiming to identify a solution that aligns with the objectives and requirements of the project.
2. Develop and test RL agents in basic environments (such as Lunar Lander [6]) with a simpler task, landing a flying vehicle, to stablish a base line and become accustomed

with the techniques before applying them in a more complex task as drone flights.

3. Implementing a custom environment that connects Liftoff with Gymnasium, allowing the interaction in between the simulator and the RL agents, with the objective of opening it to the research community and easing up its reusability for future works.
4. Train autonomous RL agents using the developed environment, with the aim of enabling them to learn how to fly a drone, follow a road, and avoid crashes. This includes tuning hyperparameters and optionally applying advanced training strategies such as imitation learning or curriculum learning.
5. Evaluate the performance of the trained agents, analysing their behaviour, learning curves, and task success rates to assess the effectiveness of the environment and training setup.

Project Planning

This chapter outlines the planning developed for the project. However, before diving into the specific timeline of the project, it is important to further specify the whereabouts of the project.

2.1 Project Scope

To better contextualize the planning strategy, this section presents a detailed breakdown of the work packages that define the project's scope. The objective is to clarify the specific tasks and responsibilities addressed throughout the development, from research to implementation, experimentation, and documentation. The project is structured into five main packages, described as follows:

1. **Research:** This stage includes a comprehensive review of the current state of the art in reinforcement learning applied to drones. Additionally, an analysis of existing drone flight simulators is conducted to identify those with suitable physical and visual realism for RL training.
2. **Development**
 - **Communication System:** This component involves the development of a bidirectional bridge between the agent and the Liftoff simulator. It includes a module to emulate RadioController commands through a virtual gamepad, and another module to capture and extract relevant visual data from the simulator's screen.
 - **Environment Development:** On one side, a Gymnasium-compatible version of the Lunar Lander environment is implemented as a baseline for early experiments. On the other side, a complete custom Gymnasium environment is developed for Liftoff, including validation to ensure correct observation-action loops and reward mechanisms.

2. PROJECT PLANNING

- **Agent Implementation:** RL agents are trained using two different frameworks. Initially, algorithms from UBC_GameIntelligence and Stable Baselines3 are used to train agents in Lunar Lander. Later, Stable Baselines3 algorithms are applied to the more complex drone flight task in Liftoff.

3. Experimentation

- **Evaluation and Analysis:** The performance and behavior of the agents are assessed in both the Lunar Lander and Liftoff environments, using various metrics and visualizations to validate their learning progress and stability.
 - **Extended Testing:** Additional experiments are carried out to explore advanced RL techniques, such as curriculum learning, imitation learning based on agent-experience alternation, and training using trajectories extracted from video.
4. **Documentation** This package includes the writing of a detailed final report summarizing the methodology, development process, and results. It also includes the preparation of presentation materials and rehearsals for the project defense to ensure clear and effective communication.
5. **Project Management** This involves the definition of a detailed work plan with milestones, timelines, and task allocations. Progress is tracked continuously, with adjustments made as needed to respond to risks or deviations from the schedule.

Table 2.1 details the estimated hours for each task to complete the project.

Package	Task	Estimated Effort (hours)
Research	State of the art research in RL for drones	25
	Research on drone flight simulators	15
Development	Communication system development	50
	Environment development (Lunar Lander and Liftoff)	50
	Agent implementation and training (Lander and Liftoff)	80
Experimentation	Evaluation and analysis of RL agents	10
	Testing curriculum and imitation learning techniques	10
Documentation	Final report writing	30
	Preparation of the defense	10
Management	Planning	10
	Monitoring and control	10
TOTAL		300

Table 2.1: Estimated effort by task group (Total: 300 hours)

2.2 Project Timeline: Gantt Chart

The temporal planning of the project is distributed across the five main work packages and spans the full duration of the academic semester. The project begins with the research phase, followed by development and experimentation, while documentation and project management activities run in parallel across the timeline.

Figure 2.1 shows the Gantt chart, detailing the estimated start and end dates for each major task. The development phase is divided into sequential subtasks, whereas experimentation and documentation are carried out in parallel to ensure efficiency and time balance.

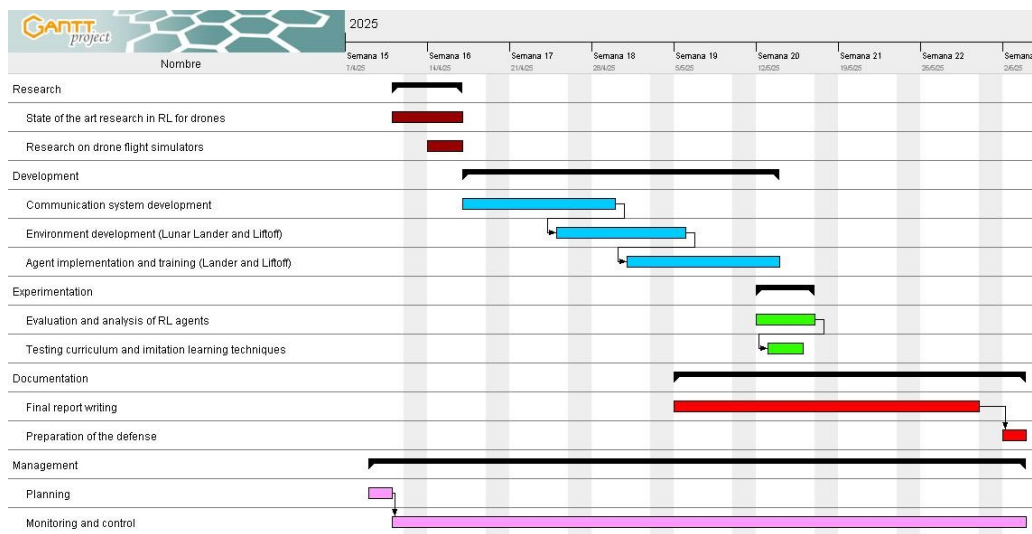


Figure 2.1: Gantt chart showing the timeline of the project tasks

2.3 Risk Management

This section identifies the potential risks that may affect the success of the project, along with their estimated probability, impact, and mitigation strategies.

1. Risk of insufficient computational resources:

- **Description:** RL algorithms may require a significant amount of computational power, especially when the state space is large or the neural networks used are deep and complex. This may lead to long training times and inefficient development cycles.
- **Probability:** Medium
- **Impact:** High
- **Prevention:** Assess the computational requirements at the beginning of the project and optimize the codebase accordingly.

- **Mitigation:** Improve code efficiency, simplify the simulation setup when possible, and consider using cloud-based GPU resources if local hardware is insufficient.

2. Risk of unstable agent training:

- **Description:** RL training is highly sensitive to hyperparameters and may result in unstable or suboptimal behavior, especially in high-dimensional or noisy environments.
- **Probability:** High
- **Impact:** Medium
- **Prevention:** Review best practices for algorithm selection and establish baseline values for hyperparameters through simpler environments.
- **Mitigation:** Perform systematic hyperparameter tuning, apply reward shaping techniques, and monitor training using visualization tools and callbacks.

3. Risk of integration complexity with Liftoff:

- **Description:** Liftoff was not originally intended for RL integration, so enabling programmatic control and observation extraction may pose unexpected technical challenges.
- **Probability:** Medium
- **Impact:** High
- **Prevention:** Review available APIs, conduct feasibility tests early, and modularize the communication architecture to isolate potential issues.
- **Mitigation:** Develop and test a minimal viable prototype as early as possible and incrementally integrate full functionality.

4. Risk of high dependence on visual extraction (OCR):

- **Description:** Relying on screen capture and OCR to extract simulation data introduces uncertainty due to potential misreadings, latency, or variability in visual outputs.
- **Probability:** Medium
- **Impact:** Medium
- **Prevention:** Use standardized screen layouts and fixed resolutions to maintain consistency in OCR input.
- **Mitigation:** Apply preprocessing techniques, implement error smoothing (e.g., moving average), and validate OCR results against known test cases.

5. Risk of delays in documentation and reporting:

- **Description:** The technical complexity of the implementation may divert attention from documentation tasks, potentially leading to incomplete or delayed reporting.
- **Probability:** Medium
- **Impact:** Medium
- **Prevention:** Define fixed periods within the project timeline for report drafting and progress summaries.
- **Mitigation:** Maintain documentation in parallel with development, using structured templates and writing incrementally throughout the project.

2.4 Monitoring and Control

To ensure the successful execution of the project, regular monitoring and control mechanisms have been implemented throughout its lifecycle. At the beginning of the project, a detailed plan was created with estimated hours, task dependencies, and milestones. Meetings with the project supervisors were scheduled every two weeks during the initial stages and increased to weekly check-ins during the core development and experimentation phases.

Project progress was tracked against the Gantt chart and time allocation table (see 2.2), allowing for adjustments in task priority and workload based on actual conditions. Additional logging of training sessions and experiment results was maintained in version-controlled notebooks to support reproducibility and traceability. If any deviation from the original timeline occurred, corrective actions such as task reordering, scope reduction, or increased documentation time were considered and applied accordingly.

2. PROJECT PLANNING

Package	Task	Estimated Effort (hours)	Actual Effort (hours)
Research	State of the art research in RL for drones	25	30
	Research on drone flight simulators	15	20
Development	Communication system development	50	60
	Environment development (Lunar Lander and Liftoff)	50	70
	Agent implementation and training (Lander and Liftoff)	80	80
Experimentation	Evaluation and analysis of RL agents	10	10
	Testing curriculum and imitation learning techniques	10	10
Documentation	Final report writing	30	40
	Preparation of the defense	10	10
Management	Planning	10	10
	Monitoring and control	10	10
TOTAL		300	350

Table 2.2: Estimated effort by task group vs actual effort taken

Background and Tools

Reinforcement Learning (RL) plays a central role in the development of this project, serving as the foundation for training agents capable of autonomous drone control. Before delving into the specific implementation details, it is essential to understand the key concepts, structures, and tools that underpin RL-based systems. This chapter offers a brief yet comprehensive overview of the fundamental components of RL, including its learning paradigm, typical algorithms, and design considerations. It also introduces several Python libraries used for implementing RL agents.

3.1 Reinforcement Learning

3.1.1 Introduction

RL is a subfield of machine learning in which an agent learns to make decisions by interacting with an environment in order to maximize a cumulative reward. Unlike supervised learning, where the model is trained on labelled data, RL relies on feedback from its own actions in the form of rewards or penalties. Over time, the agent refines its behaviour through trial and error to discover an optimal policy for acting in the environment.

The main components of RL are described below, with the interaction between the agent and the environment shown in the figure 3.1:

- **Agent:** The agent is whatever makes the decisions within the RL framework. It can take many forms depending on the context, such as a robot navigating a room, a character playing a video game, or a trading algorithm in finance. The agent observes the environment and selects actions based on its current policy, which is the strategy it uses to determine which action to take in a given state.
- **Environment:** The environment is everything external to the agent that it can interact with. It responds to the agent's actions by transitioning to new states and emitting

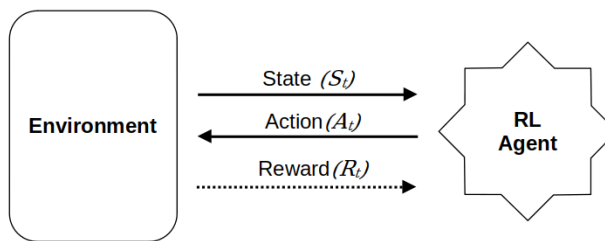


Figure 3.1: Communication between an RL agent and the environment

rewards. The environment defines the rules of the task and serves as the world within which the agent learns and operates.

- **State:** A state is a representation of the current situation of the environment. It contains all the information the agent needs to make a decision. Depending on the task, the state can be fully observable (the agent has complete information) or partially observable (the agent has limited information about the environment).
- **Action:** Actions are the choices available to the agent at each step. These can be discrete (e.g., move left or right) or continuous (e.g., set a specific motor torque). The agent selects actions in order to influence the environment and progress toward its goal.
- **Reward:** The reward is a scalar value provided by the environment after the agent takes an action. It indicates the immediate benefit of that action. The agent's objective is to learn a policy that maximizes the total reward over time, which may require balancing short-term gains with long-term outcomes.
- **Policy and Value Function:** The policy is the mapping from states to actions, determining the agent's behaviour. It can be deterministic or stochastic. The value function estimates the expected return (i.e., future cumulative rewards) from a given state or state-action pair, and is used to guide the agent's learning process. Some methods explicitly model these functions, while others, like policy-gradient methods, directly optimize the policy.

This framework of agent, environment, actions, states, and rewards is typically modelled as a Markov Decision Process (MDP), where the next state and reward depend only on the current state and action, and not on the past.

A fundamental challenge in reinforcement learning is the trade-off between *exploration* and *exploitation*. Exploitation refers to the agent leveraging its current knowledge to select

the best-known action in order to maximize reward. Exploration, on the other hand, involves trying out less certain actions to discover potentially better strategies. Balancing these two behaviours is crucial for learning an optimal policy, especially in environments where the reward structure is unknown or partially observable.

If the agent relies solely on exploitation, it may converge prematurely to a suboptimal policy, missing out on more rewarding strategies it has not yet encountered. Conversely, excessive exploration can lead to inefficient learning, as the agent wastes time trying actions that do not improve performance. Various techniques have been developed to manage this balance, such as ϵ -greedy policies, where the agent selects a random action with probability ϵ and the best-known action with probability $1 - \epsilon$, or more advanced strategies like Upper Confidence Bound (UCB) and Thompson Sampling.

Striking an effective balance between exploration and exploitation is context-dependent and often requires tuning of parameters or adaptive mechanisms that evolve over time. For instance, in ϵ -greedy methods, the value of ϵ is often decreased as learning progresses, allowing the agent to explore early on and exploit more as it gains confidence in its value estimates.

Several standard algorithms have been developed to solve reinforcement learning problems. These methods differ primarily in how they update the agent's policy and estimate future rewards. Below are two foundational algorithms widely used in RL research and applications:

- **Q-Learning:**

Q-learning is an off-policy, model-free algorithm that learns the optimal action-value function $Q(s, a)$ by iteratively updating estimates based on the Bellman equation. At each step, the agent updates the Q-value for the current state-action pair using the maximum estimated value of the next state. This encourages the agent to follow the greedy policy in the long run, even if it explores initially. The update rule is given by:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- **SARSA (State-Action-Reward-State-Action):**

SARSA is an on-policy algorithm that updates the Q-values using the action actually taken by the agent. It incorporates the agent's current policy into the update step, leading to more conservative learning compared to Q-learning. The SARSA update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma Q(s', a') - Q(s, a) \right]$$

These algorithms form the foundation for more advanced methods, such as Deep Q-Networks (DQN) and actor-critic approaches. Their simplicity makes them suitable for learning in discrete state-action spaces and for use as baselines in evaluating more complex agents.

3.1.2 Curriculum Learning

Curriculum Learning is a training strategy inspired by the way humans and animals acquire complex skills: by starting with simpler tasks and progressively increasing difficulty. In the context of RL, Curriculum Learning involves organizing the learning process in a structured way so that the agent is gradually exposed to more challenging situations as its capabilities improve [7].

This approach can significantly improve training efficiency, convergence speed, and final performance, especially in environments where solving the full task from scratch is too difficult or unlikely with random exploration.

A curriculum can be hand-crafted or learned automatically, and it typically consists of a sequence of tasks or environments with increasing complexity. It may vary based on:

- **Environment complexity** (e.g., fewer obstacles, shorter goals).
- **State initialization** (e.g., starting closer to the goal).
- **Reward shaping** (e.g., using denser rewards early on).
- **Action constraints** (e.g., limiting the agent to a subset of actions initially).

Alongside the different ways the curriculum may change, there are different strategies for implementing Curriculum Learning:

- **Manual Curriculum:** The sequence of tasks is predefined by the designer based on domain knowledge.
- **Automated Curriculum:** The curriculum is adapted dynamically based on the agent's performance, e.g., by generating new tasks that are just beyond the agent's current capabilities.
- **Teacher-Student Paradigm:** A "teacher" component generates suitable tasks for the "student" agent, often using heuristics or learning itself.

Regarding the improvements Curriculum Learning provides, it has been shown to:

- Improve sample efficiency by guiding the agent through easier tasks before attempting harder ones.
- Enable learning in tasks with sparse or delayed rewards by scaffolding the training process.
- Increase robustness and generalization by exposing the agent to a diverse range of experiences.

By leveraging Curriculum Learning, RL agents are more likely to succeed in complex environments that would otherwise be too difficult to solve through unstructured exploration alone. This method is particularly useful in domains such as robotics, navigation, and games, where task complexity can be gradually scaled.

3.1.3 Imitation Learning

Imitation Learning is a type of machine learning where an agent learns to perform a task by observing demonstrations from an expert, rather than learning through trial and error as in standard RL. It is especially useful in environments where exploration is difficult, unsafe, or too slow to achieve good results with RL alone.

In many real-world tasks, it may be easier to collect demonstrations from a human or pre-programmed expert than to design a reward function that leads to the desired behaviour. For example, teaching a robot to pour water or drive a car can be much more efficient through demonstration than exploration. Therefore, when applying Imitation Learning, the agent tries to learn a policy that mimics the behaviour of the expert, generalizing from examples and applying to new situations actions based on what it has seen in similar examples.

There are several ways to implement imitation learning:

- **Behaviour Cloning (BC):** The simplest form of imitation learning, where the agent is trained in a supervised way to copy the actions taken by the expert in each state.
- **Inverse Reinforcement Learning (IRL):** Instead of learning the actions directly, the agent tries to infer the reward function that the expert is optimizing, and then learns a policy based on that reward.
- **Dataset Aggregation (DAgger):** An interactive method where the agent learns from its own mistakes by repeatedly collecting expert corrections as it explores new states.

Imitation Learning is often used in robotics, self-driving cars, and video game agents—any scenario where expert demonstrations are available and can guide the learning process. It is also commonly used as a pretraining step before RL to give the agent a good starting point as shown in the DAgger algorithm [8].

3.2 Python libraries for Reinforcement Learning

3.2.1 Gymnasium

Gymnasium [5] is a widely-used toolkit for developing and comparing RL algorithms. It provides a standardized interface for RL environments, making it easier for researchers and practitioners to implement, evaluate, and benchmark agents across a variety of tasks. Gymnasium is a maintained fork of the original OpenAI Gym library [9] and serves as the official successor, continuing its legacy with improved support, extensibility, and community engagement.

In the RL framework, environments play a crucial role by defining the world in which the agent operates, including the action and observation spaces, dynamics, and reward structure. Gymnasium simplifies the creation and management of these environments by providing a common API that allows seamless interaction between agents and tasks. This abstraction helps decouple algorithm development from environment implementation, facilitating reproducibility and code reuse.

Each Gymnasium environment follows a simple and consistent API, which includes the following core methods:

- `env.reset()`: Initializes the environment and returns the initial observation.
- `env.step(action)`: Applies an action, updates the environment, and returns a tuple containing the next observation, reward, termination signal, truncation signal, and additional info.
- `env.render()`: (Optional) Visualizes the current state of the environment.
- `env.close()`: Cleans up resources associated with the environment.

This interface ensures compatibility with most RL libraries, including SB3, making it an essential component in modern RL pipelines. Along with the standardized api, gymnasium also includes a broad collection of benchmark environments spanning multiple domains, such as:

- **Classic Control**: Simple control problems like CartPole and MountainCar.
- **Box2D**: Physics-based tasks including LunarLander.
- **Atari**: High-dimensional visual tasks based on classic Atari 2600 games.
- **Mujoco and Robotics**: Continuous control tasks involving robotic arms and locomotion.

These environments serve as standard testbeds for evaluating RL algorithms and promoting reproducibility across studies.

Aside from the already existing environments, gymnasium supports the creation of custom environments through subclassing of its base environment class, allowing for modification of already existing environments or the creation of new ones. This makes it suitable for both academic research and real-world applications where domain-specific tasks need to be modelled. The modular design allows researchers to define tailored observation and action spaces, reward functions, and transition dynamics.

3.2.2 UBC_GameIntelligence

UBC_GameIntelligence [10] is a python library developed for research and academic purposes, implementing some of the basic RL algorithms (Q-learning Value Iteration, DQN, etc.) in a modular and easily modifiable way. The usage of this library allowed for a much simpler way to tweak and test different agents, along with helping finding and fixing a few bugs in the library itself, to then apply the best working changes in more fine-tuned and efficient libraries.

The relevant agents for the topic of this work are the reinforcement learning agents, which include the simple agents as Q-learning, or DQN and the main variants explained in the paper Rainbow[11] and a few more.

- **MultiDQN:** This proposal is an adaptation of the DQN agent for it to work with environments where the action space is MultiDiscrete, meaning that the action to take are multiple discrete values in each step. The only needed change for this variation is substituting the last layer of the DQN's neural network for a new layer with the same size as the sum of the number of values that each action requires 3.2.
- **MixedBuffer:** This variation mixes imitation learning 3.1.3 with any learning agent, as a variation of the DAgger algorithm [8]. Imitation learning tries to teach an agent to behave as an expert using pre-recorded data, and following that line along with the Replay Buffer of the DQN agent, a Mixed Buffer is defined, which contains a set of recorded data along with the experiences of the agent.
This buffer behaves just as a regular Replay Buffer, but when the agents samples experiences it will obtain a mixed set of new experiences and the pre-recorded ones. The proportions of the data will go shifting from more pre-recorded experiences to almost none, giving the agent the opportunity to learn a behaviour similar to the one recorded and then improving with it's own experience.

3.2.3 Stable Baselines3

Stable Baselines3 (SB3) [12] is a popular open-source library that provides reliable and easy-to-use implementations of state-of-the-art deep learning RL algorithms. Built on top of PyTorch [13], an open-source deep learning framework, SB3 is designed to simplify the process of training, evaluating, and deploying RL agents in both research and applied settings.

Developing RL algorithms from scratch can be complex and time-consuming, especially when handling subtle bugs and tuning hyperparameters. SB3 addresses this challenge by offering standardized, well-tested implementations of widely-used RL algorithms. This allows practitioners and researchers to focus on experimentation, environment design, and solving domain-specific problems—rather than on low-level algorithmic details.

SB3 supports several leading RL algorithms, including:

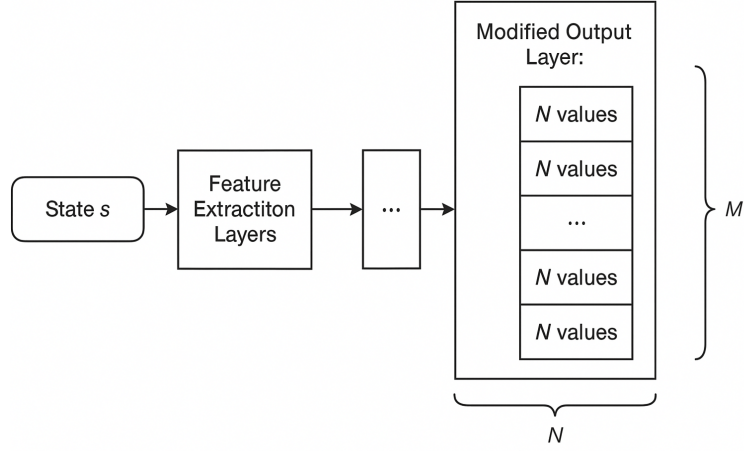


Figure 3.2: MultiDiscreteDQN schema

M: Number of discrete values for each actions

N: Possible options for each discrete values

- **PPO** (Proximal Policy Optimization): A policy gradient method that improves training stability by limiting the size of policy updates.
- **DQN** (Deep Q-Network): A value-based method that uses deep neural networks to approximate Q-values for discrete actions.
- **A2C** (Advantage Actor-Critic): A synchronous actor-critic method that estimates both value and policy to reduce variance in training.
- **SAC** (Soft Actor-Critic): An off-policy actor-critic algorithm that maximizes the trade-off between reward and entropy for better exploration mainly in continuous action spaces.
- **DDPG** (Deep Deterministic Policy Gradient): An off-policy algorithm for continuous actions that learns a deterministic policy using actor-critic architecture.
- **TD3** (Twin Delayed DDPG): An improved version of DDPG that reduces overestimation bias using twin Q-networks, delayed policy updates and target policy smoothing.

For more details on each of these algorithms, the Stable Baselines3 documentation can be consulted [14]. In addition to these algorithms, SB3 also provides an experimental extension called SB3 Contrib, which implements cutting-edge features and algorithms, including:

- **QR-DQN** (Quantile Regression DQN) A method based on DQN that makes use of quantile regression to model the distribution of the Q-values instead of the mean.

- **TQC** (Truncated Quantile Critics): An entropy-regularized actor-critic method that improves value estimation by truncating the least useful quantiles.
- **TRPO** (Trust Region Policy Optimization): A policy optimization algorithm that uses trust regions to ensure safe and efficient policy updates.
- **ARS** (Augmented Random Search): A simple and scalable evolutionary strategy for policy optimization based on random perturbations of the parameters.

All algorithms follow a unified interface, making it easy to switch between methods or compare performance across different tasks and environments.

Due to its modular design and ease of use, SB3 is widely adopted in academic research, industrial experimentation, and educational contexts as demonstrated in the work [12]. It is particularly useful for benchmarking RL algorithms, developing prototypes, and applying RL to custom or simulated environments.

Related Work

The ideal scenario for this project is to have a simulation environment that accurately models the relevant physical dynamics involved in drone flight. Furthermore, to be suitable for RL, this environment must be compatible with RL agents, providing structured observations and rewards, and accepting actions as input. This compatibility is essential for enabling effective agent training and experimentation within a high-fidelity yet accessible framework.

Previous attempts to approach this ideal scenario can be found in works such as that of Daniel González[15], where a new simulation is built in Matlab and Simulink. This work uses Unreal Engine, a graphics engine often used in video games. They are able to navigate the drone through the environment and even train an RL agent to fly it to a determined point. More simulations like this have been developed, the most well-known being AirSim [16], a simulator built on Unreal Engine that offers physically and visually realistic environments for testing autonomous vehicles. It was designed to address the high cost and complexity of real-world testing by enabling large-scale data collection and experimentation with machine learning models in simulation. AirSim includes a high-frequency physics engine, support for hardware-in-the-loop (HITL) simulations using protocols such as MavLink, and a modular architecture that allows easy extension to different vehicle types, hardware, and use cases.

However, the physics implemented in these simulations is not enough for the level of realism required in this project, as it is missing many aspects that would affect the flight if taken to the real world, such as advanced air resistance, wind effect, drone battery, drone sensor noise. Since accurately modelling all of these factors is a highly demanding task, this approach does not meet the requirements of the present work.

There are approaches where the drone has also flown in the real world. One notable example is the work of Kaufmann et al. [17], where the flight environment is highly sensorized, providing the drone with much more information about the track than it would obtain under normal conditions. Their objective is to allow a drone to race at the same level as human experts on a gate-based circuit, requiring it to fly through all gates in the shortest possible

time. To achieve this, a detailed map of the circuit is created, and the drone’s position is estimated at each moment using both its onboard camera and external sensors, allowing it to make more informed decisions about its trajectory. Furthermore, although the agent is initially trained in simulation, real-world data is later collected and incorporated into the training process to reduce the simulation-to-reality gap.

However, needing to sensorize the environment or even having a 3D map of it can be very limiting when trying to take the drone navigation out of the closed circuit racing context. This is why this option is not appropriate for this work either; therefore, another approach has been used.

The last option remaining consists of adapting an existing simulator for RL. However, unlike environments designed from the ground up for RL, existing realistic simulators are not inherently built to provide structured observations, receive action commands programmatically, or return scalar rewards, three essential components in any RL pipeline. As a result, integrating such a simulator requires addressing several challenges, including the extraction and preprocessing of state information, the mapping of discrete or continuous agent actions to in-game controls, and the definition of a meaningful reward function that reflects task objectives. Therefore, this option involves creating a communication bridge between the simulation environment and the RL framework to enable interoperability between the drone simulator and the RL agents, for which the Gymnasium interface can be used, as it is widely used in the RL research community due to its standardisation of environment-agent interactions.

To be able to make this new approach work and successfully communicate RL agents and the simulator, an appropriate selection is key; a good selection will facilitate the development of the bridge, while a bad one will not only make it harder, but, if it does not represent reality with a high enough accuracy, also negate all the benefits of this approach over the simulations already prepared for RL.

Therefore, the main feature that must be considered is the fidelity of the physical dynamics to reality, as well as including the main noise factor and alterations that may be found in a real environment. Alongside this, to facilitate the communication process and development, a friendly interface to extract information from, being able to immediately restart the environment after a crash, and including all the necessary information for the agent are also key factors to search.

Looking at these features, a few options emerge, such as Velocidrone [18], a fast and highly customisable FPV drone racing simulator; Uncrashed [19], known for its realistic flight physics and visually detailed environments; or Liftoff [4], a popular FPV simulator that offers a wide range of drones and tracks, as well as strong community and modding support. The appearance of these simulators can be observed in Figure 4.1. A detailed comparison of these simulators based on the key features mentioned above is presented in the following table.

Taking into account the characteristics of each simulator into account, **Liftoff** was chosen, because it provides imaging similar to reality and cutting-edge physics, which will be further explained in Chapter 6.

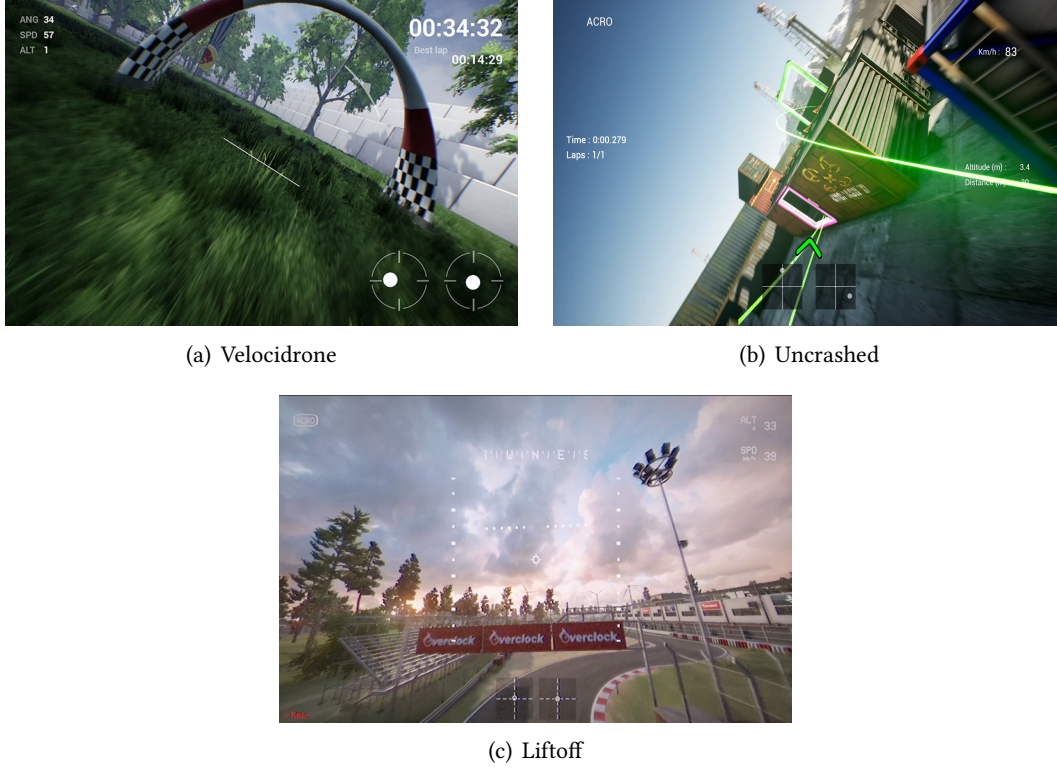


Figure 4.1: Images of the 3 considered simulators

Feature	Velocidrone	Uncrashed	Liftoff
Physical Dynamics Fidelity	Very realistic, highly configurable	Smooth, less realistic	Realistic, data-driven
Main Noise Factors	Advanced VTX noise, camera effects	Minimal noise, visual focus	Prop wash, basic noise
Environmental Alterations	Many (shadows, water, sky, etc.)	Detailed maps, some moving objects	Basic (mainly gravity)
Interface for Info Extraction	Detailed telemetry, quick access	Simple info, easy setup	Good telemetry, user-friendly
Immediate Environment Reset	Fast, customizable reset	Instant reset	Quick reset
Real-Time Observations for Agent	Access to key states via mods	Very limited; mostly visual cues	Key states exposed during flight

Table 4.1: Summary comparison of Velocidrone, Uncrashed, and Liftoff simulators.

Preliminary Experiments in Basic Environments

To begin with the experimental side of the project, some RL agents have to be tested in simpler environments first, before using them in more complex tasks as flying drones. This first step is key to establish a baseline performance and gain understanding over the training dynamics of RL. To achieve this objective, the popular Lunar Lander environment is selected, often used for tutorial and demonstrations, as it offers a controlled but challenging scene where a flying vehicle has to be landed under basic physics restrictions such as wind or gravity.

5.1 The Lunar Lander environment

The LunarLander-v3 [6] environment is a standard benchmark in reinforcement learning, provided by the Gymnasium library. It simulates the task of landing a lunar module safely on a designated landing pad (View figure 5.1), balancing the forces of gravity, inertia, and engine thrust. The agent must learn to control the lander's engines to achieve a soft landing, making it an ideal testbed for discrete and continuous control algorithms.

State Space:

The environment provides an 8-dimensional continuous state vector that captures the current dynamics of the lander:

- Horizontal position
- Vertical position
- Horizontal velocity
- Vertical velocity

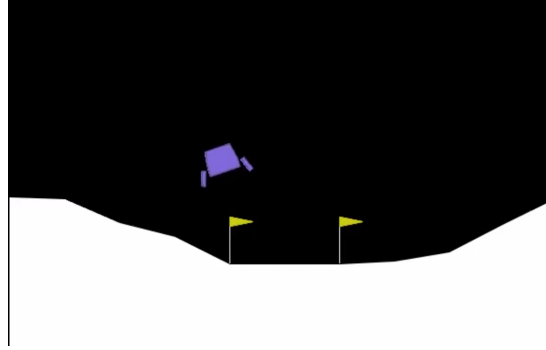


Figure 5.1: Lunar lander environment image

- Angle of the lander
- Angular velocity
- Left leg ground contact (Boolean: 0 or 1)
- Right leg ground contact (Boolean: 0 or 1)

Action Space:

In the default discrete setting, the action space consists of four actions:

- 0: Do nothing
- 1: Fire left orientation engine
- 2: Fire main engine (upward thrust)
- 3: Fire right orientation engine

Alternatively, the environment supports a continuous version using the continuous parameter, where the action is a two-dimensional vector specifying continuous thrust levels for the main and side engines.

Reward Function:

The agent receives dense feedback through a shaped reward function:

$$R = r_{\text{distance}} + r_{\text{velocity}} + r_{\text{angle}} + r_{\text{legs}} + r_{\text{fuel}} + r_{\text{terminal}}$$

- r_{distance} : Increased or decreased based on the distance to the landing pad — closer is better.
- r_{velocity} : Increased when the lander is moving slower; decreased when moving faster — encouraging smooth landings.

- r_{angle} : Decreased proportionally to how tilted the lander is — penalizes non-horizontal angles.
- $r_{\text{legs}} = 10 \times \text{num_legs_in_contact}$: A reward of +10 points per leg in contact with the ground.
- $r_{\text{fuel}} = -0.03 \times \text{side_engine_frames} - 0.3 \times \text{main_engine_frames}$: Penalty per frame of engine usage to simulate fuel cost.
- $r_{\text{terminal}} = \begin{cases} +100 & \text{if the lander lands safely} \\ -100 & \text{if the lander crashes} \end{cases}$
Final reward or penalty at the end of the episode based on success or failure. The episode ends when the lander either crashes, lands successfully, or the step limit is exceeded.

Customization Parameters:

The LunarLander-v3 environment allows limited customization directly, but more control is possible by modifying its source or using wrappers. Some key parameters include:

- `continuous (bool)`: Switches to continuous control mode when True.
- `enable_wind (bool)`: Adds lateral wind disturbances, increasing difficulty.
- `enable_turbulence (bool)`: Adds randomness to the engine outputs for more realistic dynamics.
- `gravity (float)`: Changes the gravitational force affecting descent speed.
- `max_steps (int)`: Sets the maximum number of steps per episode (default is typically 1000).

This environment is often used to benchmark discrete action algorithms such as Q-learning and SARSA, as well as advanced methods like PPO, DDPG, and SAC in continuous settings. Its variable difficulty and shaped rewards make it especially suitable for studying agent learning behaviour and generalization under controlled physics-based tasks. This makes Lunar Lander an ideal environment to test the different agents before actually training them to fly a drone, as it shares several key characteristics with the drone flying environment, such as continuous control, stability management, and goal-directed motion; but in a simplified two-dimensional setting. It provides a well-structured reward function, is relatively quick to train on, and has a lower dimensionality, making it easier to debug and tune early agent designs. By using Lunar Lander as a stepping stone, agents and hyperparameters can be validated and refined before applying them to the more complex, higher-dimensional drone environment. This reduces the risk of wasting computing resources and time in the early stages of development and allows for more informed decisions when transitioning to the target task.

5.2 Setup and training of agents

This phase consists on two sets of experiments, both carried out in the Lunar Lander environment. In the first one, the different agents implemented in the UBC_GameIntelligence library aim to be compared in between them and against their implementations in SB3. However, as most of these agents have specific constraints involving the observation and action spaces, in regards to the UBC library, only the different versions of the DQN can be used for Lunar Lander. And so, they will be trained aiming to grasp the differences on the various upgrades of the DQN and how much the implementations offered by SB3 can improve the training process.

The following agents have been tested:

- Random agent
- DeterministicDQN
- StochasticDQN
- DoubleDeterministicDQN
- DuelingDeterministicDQN
- EsperienceReplayDeterministicDQN
- StableTargetNetDeterministicDQN
- SB3-DQN

Then, after the first set of experiments, different algorithms all included in the SB3 library have been trained with the objective of understanding which of those different agents could achieve better results later on when training in the drone flying task. The following agents will be tested in both the discrete action space and the continuous one based on which can the algorithm work with, as some are compatible with continuous action spaces and some with discrete ones.

- DQN (Discrete)
- QR-DQN (Discrete)
- A2C (Discrete & Continuous)
- PPO (Discrete & Continuous)
- TRPO (Discrete & Continuous)
- SAC (Continuous)
- TD3 (Continuous)

- DDPG (Continuous)
- ARS (Continuous)

Each agent is trained for a set amount of steps, for the first set, there are three rounds, one for 10k steps, another for 100k steps and lastly for 1M steps. For the second one, the round of 1M steps was dropped out due to the amount of resources needed to train all agents in that high amount. Then, in each round, 4 environments are created with different parameters. Increasing the wind and turbulence power in each, to simulate harsher conditions.

5.3 Metrics and baseline results

For each trained agent, performance was evaluated over 500 distinct evaluation episodes, with the mean reward across these episodes recorded. This training and evaluation procedure is repeated five times under identical conditions to account for variability in training outcomes. For each group of five runs, the average reward is computed, and the mean and standard deviation of these five values are used as performance metrics.

The results for the first set of agents are presented in Table 5.1, while the results for the second set appear in Table 5.2. Videos of the best performing agents of each set of experiments can be found in their corresponding videos^{1,2}.

In both tables, the parameters G, W, and T correspond to the environment variables used in the Lunar Lander simulation: G denotes gravity, W represents wind intensity, and T indicates the level of turbulence. These parameters define the complexity and dynamics of the environment in which the agents were trained and evaluated.

5.4 Lessons learned

For the first experiment, it can be appreciated that the standard deviation values are overall high, which means that the training of the agent is not very consistent and goes well sometimes and bad other times. Aside from that, as it was to be expected, increasing the wind values decreases the overall rewards, as the environment becomes harder. Though, the most important information we take here is that as the amount of training steps increases, the algorithm provided by SB3 seems to outperform the others, meaning that the optimization provided by this library can heavily help the training process later on. In a secondary plane, observing the video¹ of the trained SB3 DQN agent, it remains floating without landing until the max step count arrives, from this behaviour, the relevance of a specific and well design reward function can be noted, as else cases like this could become frequent.

On the second set, there seems to be a difference in between algorithms, for as we can see, when the amount of training increases, some agents start to improve while others do not.

¹StableTargetNet - DQN 1: <https://youtu.be/fzpqnhyoUIA>

²TQC - QR-DQN 2: <https://youtu.be/RaiiWIHxpWE>

5. PRELIMINARY EXPERIMENTS IN BASIC ENVIRONMENTS

Agent	$G = -10$ $W = 0$ $T = 0$	$G = -10$ $W = 10$ $T = 1$	$G = -10$ $W = 15$ $T = 1.5$	$G = -10$ $W = 20$ $T = 2$
10k Training Steps				
DoubleDetDQN	-71.30 $\pm$ 21.99	-126.31 $\pm$ 19.38	-193.28 $\pm$ 38.73	-211.83 $\pm$ 13.37
DeterministicDQN	-192.84 $\pm$ 117.85	-284.45 $\pm$ 56.45	-223.36 $\pm$ 63.32	-197.36 $\pm$ 84.39
DuelingDetDQN	-91.85 $\pm$ 13.22	-134.94 $\pm$ 34.06	-174.56 $\pm$ 24.55	-224.74 $\pm$ 27.68
ExperienceReplayDetDQN	-177.58 $\pm$ 28.47	-210.28 $\pm$ 27.38	-237.54 $\pm$ 30.83	-259.84 $\pm$ 27.86
Random	-183.16 $\pm$ 1.07	-230.18 $\pm$ 2.37	-271.34 $\pm$ 2.19	-315.22 $\pm$ 2.99
StableTargetNetDetDQN	-50.84 $\pm$ 17.75	-102.83 $\pm$ 26.53	-142.15 $\pm$ 12.10	-197.20 $\pm$ 39.09
StochasticDQN	-216.00 $\pm$ 86.97	-284.25 $\pm$ 159.18	-222.64 $\pm$ 160.10	-181.64 $\pm$ 85.82
StableBaselines3DQN	-80.99 $\pm$ 59.35	-79.48 $\pm$ 107.79	-114.77 $\pm$ 33.52	-104.69 $\pm$ 38.28
100k Training Steps				
DoubleDetDQN	47.25 $\pm$ 54.36	-135.67 $\pm$ 56.01	-109.72 $\pm$ 55.41	-186.42 $\pm$ 40.11
DeterministicDQN	-119.38 $\pm$ 38.20	-169.49 $\pm$ 89.57	-194.55 $\pm$ 117.76	-248.26 $\pm$ 65.38
DuelingDetDQN	-34.70 $\pm$ 107.53	-135.33 $\pm$ 108.56	-139.88 $\pm$ 37.03	-246.97 $\pm$ 87.86
ExperienceReplayDetDQN	-181.59 $\pm$ 24.16	-230.45 $\pm$ 27.31	-349.71 $\pm$ 199.54	-283.47 $\pm$ 58.00
Random	-183.03 $\pm$ 1.34	-230.97 $\pm$ 2.51	-271.63 $\pm$ 1.87	-315.63 $\pm$ 3.48
StableTargetNetDetDQN	54.88 $\pm$ 90.77	30.16 $\pm$ 63.37	-92.48 $\pm$ 33.78	-156.92 $\pm$ 33.95
StochasticDQN	-73.88 $\pm$ 152.33	-124.33 $\pm$ 69.51	-165.94 $\pm$ 126.47	-156.17 $\pm$ 34.93
StableBaselines3DQN	133.46 $\pm$ 59.52	122.14 $\pm$ 57.51	41.15 $\pm$ 38.30	-68.03 $\pm$ 18.76
1M Training Steps				
DoubleDetDQN	23.08 $\pm$ 145.66	-71.65 $\pm$ 49.17	-128.92 $\pm$ 37.89	-184.38 $\pm$ 44.18
DeterministicDQN	-172.90 $\pm$ 79.82	-209.00 $\pm$ 19.24	-183.64 $\pm$ 36.97	-183.46 $\pm$ 72.28
DuelingDetDQN	47.38 $\pm$ 74.95	-27.81 $\pm$ 46.55	-256.92 $\pm$ 112.35	-159.36 $\pm$ 17.79
ExperienceReplayDetDQN	-202.47 $\pm$ 35.80	-190.08 $\pm$ 17.41	-245.58 $\pm$ 36.64	-296.01 $\pm$ 49.08
Random	-181.79 $\pm$ 0.55	-230.35 $\pm$ 2.23	-270.43 $\pm$ 1.99	-312.99 $\pm$ 3.34
StableTargetNetDetDQN	88.18 $\pm$ 64.24	-52.04 $\pm$ 88.83	-82.53 $\pm$ 30.46	-136.33 $\pm$ 50.30
StochasticDQN	55.38 $\pm$ 72.32	8.72 $\pm$ 49.64	-23.15 $\pm$ 38.02	-148.13 $\pm$ 39.62
StableBaselines3DQN	133.46 $\pm$ 59.52	122.14 $\pm$ 57.51	41.15 $\pm$ 38.30	-68.03 $\pm$ 18.76

Table 5.1: Performance of UBC RL agents on Lunar Lander across different reward configurations and training durations (mean $\pm$ std reward).

Looking at the results obtained, the QR-DQN and the TQC agents seem the most promising ones, mostly TQC, as it seems to hang on even in the environment with most wind. When looking at the video² demonstrating the behaviour of the agents, both seem to achieve an efficient solution to the problem.

Agent	$G = -10$ $W = 0$ $T = 0$	$G = -10$ $W = 10$ $T = 1$	$G = -10$ $W = 15$ $T = 1.5$	$G = -10$ $W = 20$ $T = 2$
10k Training Steps				
DQN	-80.99 ± 59.35	-79.48 ± 107.79	-114.77 ± 33.52	-104.69 ± 38.28
A2C	-49.57 ± 48.05	-114.58 ± 52.90	-131.96 ± 42.78	-255.13 ± 70.05
PPO	-147.62 ± 42.78	-191.04 ± 14.83	-226.11 ± 28.54	-276.28 ± 22.35
TRPO	-111.34 ± 11.58	-159.41 ± 8.78	-208.41 ± 14.80	-247.12 ± 12.33
QR-DQN	-57.06 ± 79.06	-121.88 ± 25.90	-136.24 ± 16.63	-124.30 ± 27.95
SAC (Cont.)	-80.30 ± 45.86	-117.73 ± 40.15	-130.63 ± 45.46	-187.73 ± 25.31
TD3 (Cont.)	-268.75 ± 160.87	-180.57 ± 37.59	-205.13 ± 85.17	-305.73 ± 144.01
TQC (Cont.)	-128.11 ± 126.82	-116.35 ± 47.03	-158.03 ± 75.41	-167.85 ± 37.39
TRPO (Cont.)	-130.20 ± 45.40	-198.48 ± 33.27	-226.94 ± 37.70	-301.85 ± 9.94
DDPG (Cont.)	-132.82 ± 65.97	-199.14 ± 45.75	-197.50 ± 70.43	-278.60 ± 73.34
A2C (Cont.)	-404.77 ± 304.83	-359.85 ± 231.57	-508.89 ± 187.56	-283.05 ± 129.71
PPO (Cont.)	-166.61 ± 41.21	-252.03 ± 19.84	-308.75 ± 26.26	-361.55 ± 26.82
ARS (Cont.)	-131.13 ± 1.12	-169.96 ± 1.83	-199.55 ± 4.40	-233.25 ± 5.01
100k Training Steps				
DQN	133.46 ± 56.52	122.14 ± 57.51	41.15 ± 38.30	-68.03 ± 18.76
A2C	0.55 ± 50.58	-46.90 ± 50.23	-100.09 ± 129.46	-57.61 ± 29.42
PPO	154.47 ± 38.86	33.79 ± 45.13	28.72 ± 20.22	-51.99 ± 14.85
TRPO	90.24 ± 13.73	29.06 ± 37.90	11.95 ± 18.16	-69.75 ± 22.51
QR-DQN	211.73 ± 41.57	150.65 ± 53.44	95.78 ± 28.49	-2.27 ± 25.53
SAC (Cont.)	40.89 ± 102.20	6.72 ± 99.20	-18.56 ± 81.63	35.56 ± 111.55
TD3 (Cont.)	68.18 ± 71.58	12.44 ± 87.47	-60.82 ± 132.34	-116.61 ± 45.78
TQC (Cont.)	178.59 ± 75.80	202.95 ± 22.53	164.50 ± 107.09	126.01 ± 96.13
TRPO (Cont.)	39.54 ± 48.12	22.46 ± 56.30	3.52 ± 43.42	-1.70 ± 51.73
DDPG (Cont.)	-135.21 ± 45.36	-126.46 ± 47.72	-183.84 ± 46.54	-201.69 ± 51.69
A2C (Cont.)	-216.82 ± 267.72	-122.66 ± 59.52	-301.93 ± 410.01	-172.90 ± 50.03
PPO (Cont.)	95.38 ± 9.11	64.84 ± 18.49	28.51 ± 25.90	-1.76 ± 16.12
ARS (Cont.)	-126.07 ± 11.65	-165.27 ± 2.54	-200.74 ± 3.58	-230.48 ± 5.72

Table 5.2: Performance of SB3 RL agents on Lunar Lander with a discrete or continuous action spaces across different reward configurations and training durations (mean $\pm$ std reward).

Integration of LiftOff with Gymnasium

Having completed the preliminary experimentation in simplified environments, the focus now shifts toward the real scenario of interest: training RL agents in a realistic drone flight simulator. This chapter details the design and implementation of a custom Gymnasium environment that serves as the communication interface between the Liftoff simulator and the RL agents. The aim is to enable programmatic control and observation within a high-fidelity simulation, transforming a human-oriented video game into a functional environment for reinforcement learning research.

6.1 Integration requirements for Liftoff

Firstly, it is important to note what Liftoff brings to the table, as that is the reason to build this bridge and what is going to help define the specifics:

While originally developed as a first-person drone racing game, it can serve as a valuable platform for Reinforcement Learning (RL) experimentation, particularly in areas requiring high-fidelity control and visual perception. While its primary audience is hobbyists and drone racing enthusiasts, the simulator incorporates realistic flight physics and detailed 3D environments that closely resemble the dynamics and sensory conditions encountered in real-world drone navigation. These features can make it a powerful, if unconventional, tool for training and evaluating RL agents in tasks involving precision, speed, and visual complexity.

One of Liftoff's key contributions to RL research is its simulation of realistic flight dynamics. The physics engine takes into account various real-world parameters such as inertia, thrust response, wind drag, and motor behaviour. This makes it an excellent environment for training agents that require nuanced control strategies, such as those performing aggressive maneuvers or operating at high speed. Compared to more simplified simulators, the learning

policies derived from LiftOff are likely to be more robust and better suited for sim-to-real transfer, a critical step in deploying RL agents on physical drones.

Additionally, LiftOff’s visually rich environments enable RL experiments that rely on raw visual input, such as image-based navigation, object avoidance, and target tracking. This is particularly useful for agents that incorporate convolutional neural networks or visual encoders as part of their observation space. The game-like nature of the simulator, complete with race courses and obstacle-dense maps, also encourages the development and benchmarking of agents in highly dynamic scenarios where decision-making must be both fast and precise—qualities relevant to many real-world drone applications like search and rescue or urban navigation.

While LiftOff is not a research-grade simulator in the traditional sense, it offers a unique and valuable environment for mid-to-late stage RL development. It may be less suitable for early prototyping or low-level environment customization compared to simulators like AirSim or Gazebo. However, it excels as a testbed for evaluating the performance and generalizability of agents trained in simpler settings. With the appropriate tool, LiftOff can become a bridge between abstract simulation and high-stakes, real-world deployment.

Attempting to bring as many features from LiftOff to RL, the following **requisites** are defined for the integration:

- The bridge has to allow smooth and seamless interaction between the agents and the environment, being able to capture all the information offered by the simulator and give the necessary components to the agents, as well as capturing the actions of the agents and forwarding them in a way the simulator is able to understand.
- The bridge should be consistent and flexible, it ought to be able to be used by as many different agents as possible, as well as adapt to the different tasks it may be used for, such as simply flying, following a path or looking for something specific in the environment.

6.2 Overview of the integration

The system operates as a closed feedback loop, where information flows in both directions between the LiftOff simulator and the RL agent, mediated by a custom Gymnasium-compatible environment referred to as the **gym-liftOff-bridge**. This environment enables autonomous control of the drone by an agent, even though LiftOff was originally designed for manual operation via physical gamepads, without any built-in support for programmatic interaction.

The overall schema of the bridge and the interactions between the agent and simulator can be seen in Figure 6.1.

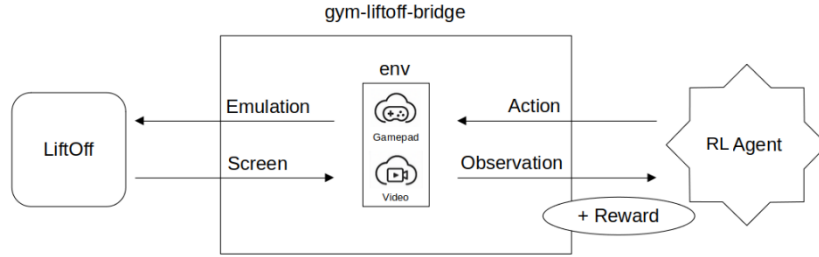


Figure 6.1: Schema of the bridge and communication between the agent and LiftOff.

At each simulation step, LiftOff produces a visual representation of the drone’s current state on screen. Since the simulator does not provide structured programmatic access to its internal data (such as position or velocity), the **Video Capturing Module** is used to capture the screen output as raw pixel data, which is then processed to extract relevant information for use by the RL agent.

Once the RL Agent receives the observation, it selects an action based on its current policy. This action, typically a vector of control values (e.g., throttle, yaw, pitch, roll), must be translated into a form that LiftOff can interpret. For this purpose, the system uses the **Gamepad Emulation Module**, which converts the agent’s output into virtual joystick signals.

In parallel, the environment computes a scalar reward value through the **Reward Function** component. This reward is based on the current observation and the agent’s behavior, and may include criteria such as proximity to a road, speed, or camera motion. The reward, along with the next observation and a termination flag, is returned to the agent to complete the standard RL loop.

6.3 Communication System

As described in the integration overview, enabling communication between the RL agent and the LiftOff simulator requires overcoming the lack of native support for programmatic interaction. To bridge this gap, the system incorporates two custom modules that facilitate bidirectional information exchange between the agent and the simulator. The following subsections describe the design and functionality of both modules in detail.

6.3.1 Gamepad Emulation Module

To address the first issue, the one of performing actions in the simulator, a gamepad is needed. To achieve this virtually, one is emulated using the library `python-uinput` [20]. This library works by simulating the USB signals that the controller would send if connected, however, these signals must be known beforehand, for what a Linux tool `evtest` has been used. This tool captures the signals sent by a device, so by connecting the physical RadioController to the computer, its values can be obtained and then be used to emulate it. The instructions obtained can be seen in the appendix A 8.

All these events have their use, but for the problem in hand, the ones most relevant are the ones that map to the movement of the drone, and so, before diving into the specific mappings, it is important to understand how drone movement is controlled. A quad-rotor drone moves through four main control inputs (see figure 6.2):

- **Throttle** controls the vertical movement of the drone. Increasing throttle makes the drone ascend, while decreasing it causes descent. It adjusts the overall speed of all motors simultaneously.
- **Yaw** controls the rotation around the vertical axis (z-axis). By varying the speeds of the diagonal pairs of motors, the drone can rotate left or right without changing altitude.
- **Roll** is the rotation around the drone's front-to-back axis (x-axis). It causes the drone to tilt left or right, allowing it to move laterally in that direction.
- **Pitch** is the rotation around the side-to-side axis (y-axis). It allows the drone to tilt forward or backward, moving in those directions.



(a) Drone movement axes: throttle (vertical), yaw (rotation), roll (side tilt), and pitch (front-back tilt). (b) Radio controller with stick inputs corresponding to drone movement axes.

Figure 6.2: Drone movement and corresponding radio controller inputs.

The next step is noting which events of the controller map to each of the axes or movement types of the drone, and then configuring Liftoff in order to assign this channels to the correct outputs of uinput events. To do that, use manual calibration, checking the channels that change with each command. The mapping of the axes, events and channels can be seen in the following table 6.1.

Axis	Event	Liftoff Channel
Throttle	ABS_Y	Channel 1
Yaw	ABS_X	Channel 0
Roll	ABS_RX	Channel 3
Pitch	ABS_RY	Channel 4

Table 6.1: Joystick Axis Mapping to Events and Liftoff Channels

6.3.2 Video Capturing Module

The second critical part of the communication system involves obtaining visual information from the simulator. To accomplish this, the system uses the PyAutoGUI library [21], which allows the user to take screenshots from the computer so that the image can be post-processed later. This tool, however, requires to know the position of the simulator on the computer's screen, along with the resolution of the screen itself to be able to capture the window correctly. To do this automatically, the user is asked to select the screen the simulator is going to be running on, and tkinter¹ is used to check the resolution of that screen. With this, it has to be noted that a display is needed, so a computer's screen has to be destined to this. Although running LiftOff in a virtual display could potentially solve this constraint, this approach has not been tested in this work, as the simulator must also remain the active foreground window for proper interaction; something that virtual displays may not reliably support.

Beyond capturing raw frames, the system also extracts additional features from each image. In particular, two key pieces of information are obtained: the drone's current speed and the position of the road or track. Speed values are extracted using OCR (Optical Character Recognition) techniques, while road features are identified through image segmentation methods, both of which enrich the agent's observations beyond pure pixel data.

To read the speed of the drone from the image, pytesseract (an OCR tool) [22] is used on the top right corner of the image, where the speed values appear. However, when trying it, it was noticed that it sometimes gave wrong lectures, mostly with the number 0, so, to improve consistency and due to the nature of the problem, a moving average system is used in each step to avoid sudden jumps in the speed values. This way, most of the errors of the OCR can be avoided and a mostly accurate speed value is obtained.

On the other hand, as the main simulation that will be used for training here is a car race track where the drone should follow the road, edge detection is done using the Canny

¹tkinter is the standard Python interface to the Tk GUI toolkit, providing a simple way to create graphical user interfaces in Python applications.



Figure 6.3: Image of Liftoff simulation

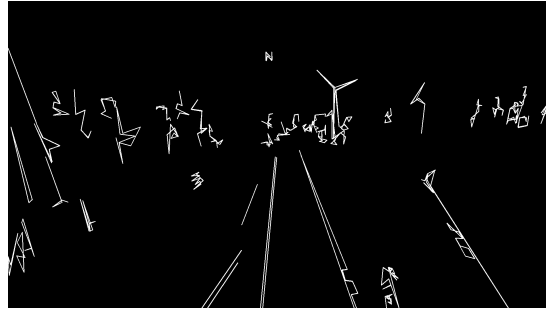


Figure 6.4: Image obtained from Canny edge detection

Edge Detection algorithm, proposed by John F. Canny [23], specifically the implementation provided by OpenCV [24]. This edge detection algorithm is applied in the image to estimate the overall position of the road, followed by applying contour detection, obtaining images such as the one shown in 6.4. Then, the biggest contour is selected, as it should be the road, some features like the centre, width and length are calculated and returned.

Finally, once all the required control mappings have been identified, a VirtualGamepad object is created using the `uinput`² library, allowing the system to emit control actions programmatically. Unfortunately, a new problem shows itself with this approach: Liftoff will only receive the information of the controller if the simulator's window is currently the active (focused) window on the computer. This constraint will bring difficulties when attempting to parallelize the training of an agent or simply using the computer for any other task while the training process is running.

²`uinput` is a Linux kernel module that allows user-space applications to create and inject input events (e.g., from a keyboard, mouse, or joystick) into the input subsystem.

6.4 Environment Design

Once the communication pipeline with the simulator is established, the next step is to define the structure of the custom Gymnasium-compatible environment that enables interaction between the RL agent and LiftOff. This environment must comply with the standard interface expected by RL frameworks, providing a well-defined state and observation space, action space, and reward function, which in this case are defined as follows:

- **State:** It is of all the information contained in the simulation in a fixed instant. Both the shown information (image) and every piece of information inside the simulator (wind computations, the position of every object inside of the map...).
- **Observation:** It is the frame captured in an specific instant. Containing the raw image of the simulation environment turned to a greyscale.
- **Actions** It is defined by 4 values, each of them being a number between 0 and 2047. Each value represents one possible control from the gamepad (throttle, yaw, roll, pitch). The range of numbers is defined by the possible values of the selected gamepad.
- **Reward** In the most basic form of the environment, it is really simple: 1 while the simulation is running and -100 when it ends.

$$r_t = \begin{cases} 1, & \text{if alive} \\ -100, & \text{if dead} \end{cases}$$

The only missing piece for the environment is being able to detect when an episode ends. In Liftoff, when the drone crashes or the flight is finished the screen briefly goes black to start over again, to detect that, the mean of all the pixels in the image is calculated, and if it is lower than 30, the episode is considered as ended and a new one is started. To ensure the episode restarts at that moment, the keyboard letter 'R' is pressed with the PyAoutoGUI library, as, when doing so, Liftoff sets the drone to the initial pose.

With this environment definition, the information obtained by the agent and the flexibility of the environment would be quite narrow, however, gymnasium offers a tool to add optional modifications to any environment, wrappers. Wrappers act like a subclass of the environment, where some part of it can be modified or redefined. The main things that can be changed like this are the reward function, the action space or how the environment behaves when an episode starts and concludes. Moreover, and what makes this format a truly powerful tool, different wrappers can be applied one on top of each other to tune the environment to the needs of the problem that is being tackled.

To meet the objectives set for this Gymnasium environment, some specific wrappers have been designed and implemented, some of which will help with compatibility and some of which will add up to the information the drone receives from the simulator.

6.4.1 Reward wrappers

Making use of this tool, it is easy to create different reward functions that seek to optimize different behaviours, or to design rewards that are gradually more complex to train an agent using curriculum learning.

These are the different defined reward functions:

- **Stability:** This function applies a penalization factor to sudden movements on the camera, making the flight more stable. Given a reward calculated on a step, it is updated as shown in the following equation:

$$updated_reward = max(reward - 0.01 \times mean(|previous_image - image|), 0)$$

- **Centred road:** In this reward, the calculated information of the road is taken. The displacement of the centre on the horizontal plane is used to give a bonus reward. The more centred the road, the bigger the computed reward.

$$updated_reward = reward + 0.1 \times (1 - horizontal_displacement)$$

- **Speed reward:** Here, a bonus reward is given in proportion to the speed of the drone. A higher speed value will result in a higher reward.

$$updated_reward = reward + 0.1 \times drone_speed$$

6.4.2 Action wrappers

Aside from the wrappers defined to tweak the reward functions, wrappers for the action space can standardize or transform the environment's native action format, which makes it easier to integrate with various RL algorithms and tools that have specific input requirements. This abstraction layer ensures that agents expecting, for instance, discrete or normalized actions can interact with the environment seamlessly, without needing to modify the core environment logic.

With this in mind, the following wrappers been created:

- **Normalized actions:** This wrapper changes the range of each value of the action space from 0-2047 to -1-1. This change sets 0 as the centre point, which can be helpful for faster training, and allows a seamless transition between different controllers that could have different value ranges.
- **Float actions:** This wrapper simply changes the data type from integer to float. This is just for compatibility with some algorithms that require the values to be of type float.
- **Discretized actions:** This wrapper changes the action space to change it from a continuous space to a set of discrete values. It takes as a parameter the amount of discrete values for each action and sets the action space as Multi Discrete, which means that it has 4 different discrete values.

6.4.3 Observation wrappers

Sometimes, there may be an interest on changing the observation so that it consists on the past n frames instead of just the current one, this may allow the agent to deduce the trajectory of the drone and its current momentum.

6.4.4 Other wrappers

Gymnasium also allows wrappers that modify different parts of the environment from the ones above. For this problem, a really useful thing to add could be taking off from the ground.

Whenever a flight is started, in Liftoff the drone has to be "armed", this means simply setting the throttle to 0 to stop the drone from taking off instantly whenever the simulation loads. Once the drone is armed, it can then start flying. This arming process is included in the bridge, at the start of each episode the gamepad values are reset and then the episode starts. However, taking off requires specific values on the gamepad, keeping everything centred except for the throttle, which should be slightly increased. This can be automated and it is in a wrapper, called **AutoTakeOff**, which changes the function that resets the episode so that the drone takes off and afterwards the episode starts.

Experiments in the Custom Environment

Now that the environment is ready and the previous experiments on Lunar Lander have been carried out, everything is ready in order to start training agents on the actual Liftoff simulator. To do so a few experiments will be carried out, starting with simple trials using pre-configured agents and default parameters, and progressively advancing to more sophisticated approaches such as extracting experiences from recorded gameplay videos and combining Reinforcement Learning with Imitation Learning techniques.

7.1 Train setup and first try with PPO

7.1.1 Setup and training of the agent

The first experiment is designed to test that the environment works correctly and the training can be carried out without issues. To do so, the PPO agent is used. Even though the TQC and QR-DQN agents got better results in Lunar Lander, they are only available in the experimental branch of the SB3 library, whereas PPO is included in the stable release, making it a safer option for the initial testing. Using PPO ensures that if the environment is working correctly, no errors are going to arise from the implementation of the agent. Starting from this experiment, the AutoTakeOff wrapper is applied to ensure the drone consistently takes off from the ground, bypassing the need for the agent to learn this behaviour and allowing training to focus directly on the flight phase.

7.1.2 Results and lessons learned

Once the Agent had trained for around 20 hours, the result can be straight up seen in the simulator, while it is still training. However, when looking at it, the agent did not seem to

have learned, as the episodes are still short and the values of the actions taken seem random. This can be because of how slow the process to create new experiences is, as one episode can take around 10 seconds (if it is a short one), and it can not be parallelized in a single machine. This comes together with the fact that PPO is an on-policy algorithm, which means that it needs data collected by the current policy to update it and cannot use data collected on previous updates. Therefore, off-policy algorithms could be better for this specific scenario where creating experiences is so costly.

7.2 Training an A2C agent

7.2.1 Setup and training of the agent

Before jumping to the more advanced and more experimental algorithms, as PPO did not seem to learn at all, and A2C agent was tested. The A2C agent is simpler and it can be slightly more stable than PPO sometimes, so it could be able to improve in some cases the behaviour of PPO, even though it will not most of the times. A few tries were made, leaving the agent training for around 20 hours between each different try.

7.2.2 Results and lessons learned

Most of the observed behaviours were closely similar to the one presented by PPO, seeming random. However, on one occasion the model learned to land by lowering the throttle value and maintaining the others centred. This is an effective way of gaining reward in the current setup, as it increases with the time the simulation is running and depending on how still the camera is. To avoid it, the proposed wrapper corresponding to giving reward based of the speed of the drone could be used once the drone starts getting higher rewards, incentivizing it keep moving instead of staying still. This agents behaviour is demonstrated in the following video¹.

7.3 Using an off-policy agent (DDPG)

7.3.1 Setup and training of the agent

To fix the problem of slow learning due to slow experience generation, an off-policy agent can be used. This agents do not require to use exclusively data obtained from the last policy to update themselves, speeding up the training proceeds for our case. The first experiment is done with the DDPG agent, as it is included in the SB3 library and will not incur extra work or errors even if the performance is not as good. Aside from testing it as in the previous experiments, a multi-thread setup was also tested, where 2 policies are stored and one thread updates one policy constantly while the other one plays an epoch, adds the new data to the experience batch and then synchronizes its policy with the other thread's. However, as SB3 is not prepared for this kind of approach, the most outer layer of the library has to be avoided

¹A2C agent landing on Liftoff: https://youtu.be/1oS_uv6Ymks

and the local policy of the agent accessed directly, leading to a more complex training and a new `predict` function having to be defined for the agent, as well as the necessary thread locks to ensure the parameters are updated correctly and the new data generated by the playing agent is correctly given to the training agent on their synchronization. In this experiment, some additional control sequences were defined too, to be able to pause, resume and stop the training, as that allows for leaning the agent training, pausing the training to work on the computer and once it is free again resume it.

7.3.2 Results and lessons learned

When reviewing the progress of the agent in the span of a day, the same result as the one in the previous experiment appeared. However, when looking at the count of updates, it was much higher than with the PPO or A2C, reducing the impact that gaining experiences at such a slow rate has on the training of the agents. This will still be a problem as the changes of behaviour caused by the updates on the policy will take a long time to populate the buffer of experiences, causing the training to still be slow. For this reason, giving a head-start to the agent in form of imitation learning could be a good way to accelerate the training, for which the previously introduced Mixed Buffer, or the DAgger[8] algorithm could be used.

When launching this experiment, if the agent was trained for long periods of time, performing around 3.5 million updates, a Segmentation Fault error appeared, which is an error that appears in C when something related to the RAM memory goes wrong. Many things were checked with this error: if locks between the threads were not correct, if infinities or *NaNs* were appearing in the policy weight values, saving the agent and loading it back again in a fresh setup..., but the problem was not found and remains there, limiting the amount the agent can be trained before failing.

7.4 Adding recorded data to the training

7.4.1 Setup and training of the agent

This step in the experimentation requires a bit more work than the previous ones. Firstly, the Mixed Buffer is going to be used, including both previously recorded data and new experiences. To do so, a new SB3 buffer class must be implemented, specifying how the buffer must store and sample the experiences through the SB3 interface. Once implemented, the experiences have to be created:

To generate experiences, the agent will learn how to fly, different videos were recorded, where a drone is flown steadily and the inputs of the controller are recorded at the same time. In this way, a video is obtained, along with time each input was performed. However, these two pieces have to be synchronised, as two different tools are being used to measure them, so an extra effort has to be made to make both inputs start at the same time.

With the video and the evtest recording at hand, that video has to be processed in order to obtain image-action pairs. To do so, the initial time of the recording is taken and from there,

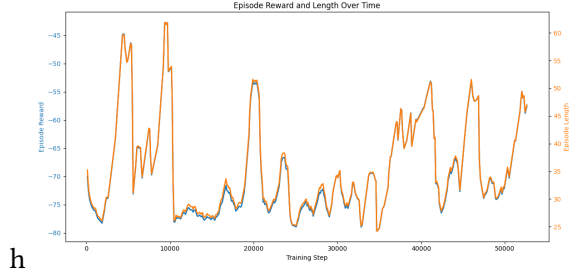


Figure 7.1: Reward and episode reward evolutions on TQC agent training

for each frame of the video (recorded at 30 or 60 fps depending on which is taken), time is advanced to meet the time of that frame. Then, the joystick inputs recorded by evtest up to that point are taken, getting at the end data looking like: Image, (throttle, pitch, yaw, roll), which is stored in a CSV file.

Finally, when the agent is created, a Mixed Buffer is assigned, and all the experiences are created from the CSV file, adding them to the imitation learning side of the buffer. Then, when the agent samples from the buffer, it will get many experiences from the videos, an amount that will gradually decrease to leave more room for the agents' own experiences. To test this method, first DDPG was used to check the correct working of the buffer and SB3 agents, and then it was applied with the TQC agent.

7.4.2 Results and lessons learned

In general, after some hours of training, the drone did not learn to fly correctly. However, it learnt that by selecting high values for all the actions, it started spinning, sometimes upward, and was able to survive a bit longer, which can be interpreted as a local optima to which the agent has arrived. This can be seen in the graph shown in the figure 7.1, where the close dependency between the reward and the length of the episode is also seen.

This result shows that the agent is starting to learn, even though it is still far from finished learning. The described behaviour of giving really high values for the actions has started showing at approximately the step 7000, and, up to when the agent has been stopped, it remained there. From this local optima and the match that can be seen between the reward and episode length, one can get the intuition that the reward function can be modified to improve the training.

7.5 Adding road and speed rewards

7.5.1 Setup and training of the agent

The last performed experiment involves adding the road and speed wrappers to enhance the reward function, this could help avoid the spinning behaviour and guide the agent to fly

watching the road in a more stable way. For the rest, the agent is the same as in the previous experiment.

7.5.2 Results and lessons learned

7.6 Overall conclusions of the experiments

Conclusions and Future Work

This project has successfully developed a Gym-compatible environment that allows reinforcement learning (RL) agents to interact with the high-fidelity drone simulator Liftoff, a commercial Steam game. Through the implementation of a bidirectional communication system — consisting of visual state extraction and virtual gamepad-based control — the simulator, originally designed for human input, was repurposed as a research-grade training platform for autonomous drone agents. This represents a significant contribution by allowing experimentation with state-of-the-art RL algorithms in drone flights in a more physically realistic setting than before, minimizing the existing sim-to-real gap.

To the best of our knowledge, no previous work has been published that integrates a commercial Steam game with Gym-based RL frameworks for autonomous control. While other simulators such as AirSim, CARLA, or Unity ML-Agents offer open interfaces for RL research, they are specifically designed with programmatic access in mind. In contrast, Liftoff presents unique challenges due to its closed nature and real-time constraints. Overcoming these barriers highlights both the technical complexity and the originality of this work.

Initial experiments using PPO and A2C highlighted the difficulties in learning drone control from scratch, due in large part to the sparse rewards and limited exploration in early stages. Switching to off-policy algorithms like DDPG enabled more efficient training by decoupling experience generation and policy updates, though training remained slow due to the long time required for meaningful feedback to accumulate. Additionally, efforts to use multithreading for parallel experience collection, while promising in concept, revealed challenges with SB3's internal structure and stability issues.

To address the slow training dynamics, a novel buffer system was designed that mixes recorded human trajectories with agent-generated experience. This approach aligns with imitation learning and curriculum strategies, and it showed potential in accelerating early learning, particularly when using the TQC agent. Although the segmentation fault that emerged during longer runs still imposes a technical limitation, the architecture is extensible

and has demonstrated promise.

Regarding how this work could be continued, there is still lots to be done in many areas, as this was a project to be rounded up in a limited amount of time with a limited amount of resources, many aspects of the bridge itself, as well as the training and testing of the agents have not been as well polished as it could have been done. Therefore, the following directions are proposed for future exploration and development:

- **Complete and evaluate imitation-enhanced training:** The preliminary implementation of a mixed replay buffer combining human demonstrations with agent experience (inspired by DAgger and imitation learning) should be further developed. Specifically, completing the experimentation with the TQC agent and performing ablation studies could clarify how different proportions of recorded vs. self-generated data influence learning stability and generalization.
- **Improve stability and performance of the communication system:** While the bridge currently supports basic training, some instability issues such as segmentation faults have been identified, particularly in long training runs. Future work should address these robustness issues and consider redesigning the architecture for better parallelism, such as enabling support for vectorized environments or asynchronous sampling.
- **Extend reward shaping and curriculum learning:** The current reward design is relatively sparse and primarily penalizes crashes. Designing more informative reward functions—such as altitude maintenance, smooth control, or waypoint tracking—could facilitate faster convergence. Introducing curriculum learning, where agents start with simpler tasks and progressively face more complex scenarios, may also significantly enhance training efficiency.
- **Sim-to-real transfer and domain adaptation:** A long-term goal is to assess whether agents trained in Liftoff can generalize to physical drones. This would involve evaluating transfer learning strategies, such as domain randomization, sensor noise modelling, or fine-tuning on real-world data. Alternatively, trained policies on more research-oriented simulators like AirSim could be tested and fine-tuned in Liftoff to bridge the gap before deployment on hardware dodging most of the problem of slow training and low parallelization capabilities.
- **Broaden task variety and environment features:** The current setup focuses on basic flight and obstacle avoidance. Expanding the environment to include task-based objectives (e.g., following a moving target, navigating a maze, or collecting visual cues) would increase its versatility for more complex RL challenges.
- **Open-source release and community validation:** Packaging the environment as an open-source Gymnasium-compatible library is planned and will allow other researchers to benefit from and contribute to the project. Providing detailed documentation, example

agents, and baseline benchmarks would help establish it as a reusable tool in the RL and robotics community.

By continuing in these directions, this work could evolve into a robust platform for realistic drone control training, helping to bridge the gap between simulation and real-world autonomous aerial systems.

Appendix

Appendix A: evtest Output

Input driver version is 1.0.1

Input device ID: bus 0x3 vendor 0x1209 product 0x4f54 version 0x111

Input device name: "OpenTX RM TX16S Joystick"

Supported events:

Event type 0 (EV_SYN)

Event type 1 (EV_KEY)

Event code 304 (BTN_SOUTH)
Event code 305 (BTN_EAST)
Event code 306 (BTN_C)
Event code 307 (BTN_NORTH)
Event code 308 (BTN_WEST)
Event code 309 (BTN_Z)
Event code 310 (BTN_TL)
Event code 311 (BTN_TR)
Event code 312 (BTN_TL2)
Event code 313 (BTN_TR2)
Event code 314 (BTN_SELECT)
Event code 315 (BTN_START)
Event code 316 (BTN_MODE)
Event code 317 (BTN_THUMBL)
Event code 318 (BTN_THUMBR)
Event code 319 (?)
Event code 704 (BTN_TRIGGER_HAPPY1)
Event code 705 (BTN_TRIGGER_HAPPY2)
Event code 706 (BTN_TRIGGER_HAPPY3)
Event code 707 (BTN_TRIGGER_HAPPY4)
Event code 708 (BTN_TRIGGER_HAPPY5)
Event code 709 (BTN_TRIGGER_HAPPY6)
Event code 710 (BTN_TRIGGER_HAPPY7)
Event code 711 (BTN_TRIGGER_HAPPY8)

Event type 3 (EV_ABS)

Event code 0 (ABS_X)	Value 1024	Min 0	Max 2047	Fuzz 7	Flat 127
Event code 1 (ABS_Y)	Value 1024	Min 0	Max 2047	Fuzz 7	Flat 127
Event code 2 (ABS_Z)	Value 1197	Min 0	Max 2047	Fuzz 7	Flat 127
Event code 3 (ABS_RX)	Value 1024	Min 0	Max 2047	Fuzz 7	Flat 127
Event code 4 (ABS_RY)	Value 0	Min 0	Max 2047	Fuzz 7	Flat 127
Event code 5 (ABS_RZ)	Value 0	Min 0	Max 2047	Fuzz 7	Flat 127
Event code 6 (ABS_THROTTLE)	Value 0	Min 0	Max 2047	Fuzz 7	Flat 127

Event type 4 (EV_MSC)

Event code 4 (MSC_SCAN)

Bibliography

- [1] Ridha Guebsi, Sonia Mami, and Karem Chokmani. Drones in precision agriculture: A comprehensive review of applications, technologies, and challenges. *Drones*, 8(11):686, 2024. See page 2.
- [2] Imen Zrelli, Abderahman Rejeb, Rawan Abusulaiman, Raghad AlSahafi, Karim Rejeb, and Mohammad Iranmanesh. Drone applications in logistics and supply chain management: a systematic review using latent dirichlet allocation. *Arabian Journal for Science and Engineering*, 49(9):12411–12430, 2024. See page 2.
- [3] Narek Papyan, Michel Kulhandjian, Hovannes Kulhandjian, and Levon Aslanyan. Ai-based drone assisted human rescue in disaster environments: Challenges and opportunities. *Pattern Recognition and Image Analysis*, 34(1):169–186, 2024. See page 2.
- [4] LuGus Studios. Liftoff: Fpv drone racing. <https://www.liftoff-game.com/liftoff-fpv-drone-racing>, 2025. Accessed: 2025-04-11. See pages 2, 22.
- [5] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2024. See pages 2, 15.
- [6] Oleg Klimov. Lunar lander environment in gymnasium. https://gymnasium.farama.org/environments/box2d/lunar_lander/. See pages 2, 25.
- [7] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pages 41–48. ACM, 2009. See page 14.
- [8] Stefano Ross, Geoffrey Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 627–635, 2011. See pages 15, 17, and 45.
- [9] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016. See page 15.
- [10] Inigo Lopez Gazpio. Ubc_gameintelligence: Repository to collect simple baselines and environments for use in academia and research. https://github.com/ilopezgazpio/UBC_GameIntelligence, 2025. Accessed: 2025-06-03. See page 17.
- [11] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning, 2017. See page 17.

BIBLIOGRAPHY

- [12] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. See pages 17, 19.
- [13] Jason Ansel, Edward Yang, Horace He, and et al. Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, April 2024. See page 17.
- [14] Documentation for stable baselines3 python library. <https://stable-baselines3.readthedocs.io/en/master/index.html>. See page 18.
- [15] Daniel González Rodríguez. Simulación del vuelo de un dron en un entorno virtual. 2022. See page 21.
- [16] Shital Shah, Debadeepta Dey, Chris Lovett, and Ashish Kapoor. Airsim: High-fidelity visual and physical simulation for autonomous vehicles. In *Field and Service Robotics: Results of the 11th International Conference*, pages 621–635. Springer, 2018. See page 21.
- [17] Elia Kaufmann, Leonard Bauersfeld, Antonio Loquercio, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. Champion-level drone racing using deep reinforcement learning. *Nature*, 620(7976):982–987, 2023. See page 21.
- [18] Bat Cave Games Limited. Velocidrone: Fpv drone racing simulator. <https://www.velocidrone.com/>, 2025. Accessed: 2025-04-28. See page 22.
- [19] Louis Rochette. Uncrashed: Fpv drone simulator. https://store.steampowered.com/app/1682970/Uncrashed__FPV_Drone_Simulator/, 2021. Accessed: 2025-04-28. See page 22.
- [20] Tuomas J. Räsänen and Gonçalo Pereira. python-uinput: Pythonic api to linux uinput kernel module. <https://pypi.org/project/python-uinput/>, 2024. Accessed: 2025-04-30. See page 36.
- [21] Al Sweigart. Pyautogui: Cross-platform gui automation for python. <https://pyautogui.readthedocs.io/>, 2023. Accessed: 2025-04-30. See page 37.
- [22] Samuel Hoffstaetter and Matthias Lee. pytesseract: A python wrapper for google’s tesseract-ocr engine, 2025. Accessed: 2025-04-30. See page 37.
- [23] John Canny. A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-8(6):679–698, 1986. See page 38.
- [24] Gary Bradski. Opencv: Open source computer vision library. *Dr. Dobb’s Journal of Software Tools*, 2000. See page 38.